

Prompt: Estoy medio confundido con las clases abstractas. En Entidad tenemos `actuar()` y `mostrarEstado()` sin código, pero `envejecer()` sí tiene código. Me explicás por qué algunos métodos son abstractos y otros no, en nuestro proyecto?

Respuesta:

Explicando cuándo un método debe ser abstracto o concreto.

Es una muy buena pregunta, y la respuesta tiene una regla simple.

La regla

Para cada método, preguntate: ¿existe un código que sirva igual para todas las entidades?

- Sí → se escribe una sola vez en la clase padre y todos lo heredan. Es un método concreto, como `envejecer()`.
- No, cada una lo hace distinto → se deja abstracto, sin código, y cada clase hija escribe su propia versión. Es el caso de `actuar()` y `mostrarEstado()`.

Aplicado a nuestro proyecto

`envejecer()` es igual para todos. Una planta, un conejo y un lobo envejecen exactamente de la misma forma: suman 1 a la edad y pierden 2 de energía. Por eso se escribe una sola vez en Entidad:

```
java
public void envejecer() {
    edad++;
    setEnergia(energia - 2);
}
```

Ni Planta, ni Conejo, ni Lobo tienen que escribirlo: lo heredan tal cual. Si mañana quisieran que `envejecer` cueste 3 de energía, lo cambian en un solo lugar.

`actuar()` es distinto para cada uno. Mirá qué hace cada clase en su `actuar()`:

Clase	Qué hace en <code>actuar()</code>
-------	-----------------------------------

Planta	Intenta reproducirse
Conejo	Busca una planta y la come
Lobo	Intenta cazar un conejo

No hay un código que sirva para los tres. ¿Qué escribirías en Entidad? Si pusieras "comer una planta", el lobo comería plantas. Por eso Entidad solo dice "toda entidad actúa, pero cada una sabe cómo":

java

```
public abstract void actuar(Ecosistema eco);    // sin llaves, termina en ;
```

`mostrarEstado()` también es distinto para cada uno, porque cada entidad muestra información diferente:

- la Planta muestra su tamaño;
- el Conejo avisa si está en peligro;
- el Lobo muestra sus cacerías exitosas.

¿Y por qué no dejarlo con llaves vacías, `{ }`?

Porque un método abstracto es una obligación. Si Conejo "se olvida" de escribir `actuar()`, Java no lo deja compilar. Con un método vacío, el conejo no haría nada en cada turno y nadie se daría cuenta del error.

Además, como Entidad tiene métodos sin código, no se pueden crear objetos de Entidad (`new Entidad()` da error). Tiene sentido: no existe "una entidad" suelta, existen plantas, conejos y lobos.

El mismo patrón se repite en Animal

Animal tiene exactamente la misma lógica:

- `comer()` es abstracto, porque el conejo come plantas y el lobo caza conejos: cada uno lo hace distinto.
- `moverse()` es concreto, porque los dos animales se mueven igual: imprimen que se desplazaron. Se escribe una sola vez.

Prompt 2: En el constructor de entidad usamos `setEnergia(energia)` en vez de hacer directamente `this.energia = energia`. hay alguna diferencia y por qué sería mejor hacerlo así en este caso?

Respuesta:

Explicando la diferencia entre asignación directa y validación con `setEnergia`.

Explicando la diferencia entre asignación directa y validación con `setEnergia`.

Sí, hay una diferencia importante: la validación.

Qué hace cada forma

Asignación directa: guarda el valor tal cual venga, sin revisar nada.

```
java
this.energia = energia;
```

Con el setter: antes de guardar, pasa por los controles que escribimos.

```
java

public void setEnergia(double energia) {
    if (energia < 0) {
        energia = 0;        // negativa → 0
    }
    if (energia > 100) {
        energia = 100;      // más de 100 → 100
    }
    this.energia = energia;
}
```

Un ejemplo concreto

Supongamos que alguien crea un conejo con un valor inválido:

```
java
Conejo c = new Conejo("Blas", -20);
```

En el constructor...	La energía queda en
<code>this.energia = energia;</code>	-20 (un conejo con energía negativa, algo imposible)

```
setEnergia(energia);      o (el setter lo corrige)
```

Lo mismo con `new Conejo("Blas", 150)`: con el setter, la energía queda en 100.

Por qué es mejor así

1. El objeto nunca nace en un estado inválido. Los setters protegen la energía cada vez que cambia durante la simulación. Pero si el constructor asignara directo, habría una "puerta trasera": al crear el objeto se podría saltar la validación. Usando el setter también en el constructor, la regla se cumple siempre, desde el primer momento.
2. La regla está escrita en un solo lugar. Si en el constructor repitiéramos los `if`, tendríamos la misma validación en dos lugares. Y si mañana cambian el tope de 100 a 150, habría que acordarse de cambiarlo en los dos. Con el setter, se cambia en uno solo.

Un detalle para ser honestos

En nuestro proyecto, hoy nadie crea entidades con valores inválidos:

- las energías iniciales son aleatorias dentro de rangos correctos;
- las crías nacen con 50 (plantas) o 30 (conejos);
- cuando el usuario elige la energía en la intervención, el `Main` solo acepta valores entre 10 y 100.

O sea que, en la práctica, el resultado hoy sería el mismo. El setter en el constructor es una protección: garantiza que la clase se cuide sola, sin depender de que quien la usa pase siempre valores correctos. Justamente eso es el encapsulamiento.

¿Y por qué edad y viva se asignan directo?

java

```
this.edad = 0;  
this.viva = true;
```

Porque no vienen de afuera: son valores fijos que escribimos nosotros. Toda entidad nace con edad o y viva, así que no hay nada que validar. El setter solo tiene sentido para los datos que llegan como parámetro: `nombre` y `energia`.

Prompt 3: me puedes explicar para que usamos la interfaz reproducible? No termino de entender por qué no pusimos los métodos de reproducción directamente en Entidad.

Respuesta:

Explicando por qué no aplica la herencia dentro de Entidad.

La clave está en qué pasaría si **sí** los pusiéramos en Entidad. Veamos las opciones y por qué ninguna funciona bien.

Opción 1: ponerlos en Entidad

Todo lo que está en Entidad lo heredan **todos**, incluido el Lobo. Si Entidad tuviera `reproducirse()`, el Lobo también "sabría reproducirse". Pero la consigna dice claramente: *"Lobo no implementa Reproducible — los lobos no se reproducen en esta simulación"*.

Para arreglarlo, habría que escribir en Lobo algo como:

```
java
```

```
@Override
```

```
public boolean puedeReproducirse() {
```

```
    return false; // "sé reproducirme, pero nunca puedo"
```

```
}
```

Técnicamente funciona, pero es un parche. El Lobo tendría un método que no le corresponde, y en el Ecosistema los lobos entrarían al recorrido de reproducción solo para responder "no". El diseño estaría diciendo algo falso sobre el lobo.

Opción 2: ponerlos en Animal

Tampoco sirve, por dos motivos:

- el Lobo es un Animal, así que tendríamos el mismo problema de antes;
- la Planta **no** es un Animal, así que se quedaría sin reproducción.

El problema de fondo

Mirá el árbol de herencia:

Entidad

|— Planta ← se reproduce

|— Animal

|— Conejo ← se reproduce

|— Lobo ← NO se reproduce

Planta y Conejo están en **ramas distintas**. El único padre que tienen en común es Entidad, y ese padre también es del Lobo. **Con herencia no hay un lugar donde poner la reproducción que le llegue solo a Planta y a Conejo.**

La solución: una interfaz

La herencia responde **qué es** cada clase. La interfaz responde **qué puede hacer**, sin importar dónde esté en el árbol. Por eso se la "pegamos" solo a quienes la necesitan:

java

```
public class Planta extends Entidad implements Reproducible { ... }
```

```
public class Conejo extends Animal implements Reproducible { ... }
```

```
public class Lobo extends Animal { ... } // sin Reproducible
```

Así el diseño dice la verdad:

- Planta **es una** Entidad y **puede** reproducirse.
- Conejo **es un** Animal y **puede** reproducirse.
- Lobo **es un** Animal y **no puede** reproducirse, porque ni siquiera tiene esos métodos.

Y además permite el polimorfismo del turno

Como Planta y Conejo comparten la interfaz, se pueden guardar juntos en una lista del tipo Reproducible:

```
java
```

```
ArrayList<Reproducible> reproducibles = new ArrayList<>();
```

En esa lista **solo pueden entrar** los que implementan Reproducible. Si alguien intentara meter un Lobo, Java daría error de compilación. Así, el recorrido de reproducción del turno procesa exactamente a quienes corresponde, sin preguntar ni filtrar nada.

Prompt 4: cómo debería funcionar Mortal? si una interfaz no tiene el atributo energía, como puede verificarMuerte() saber cuánta energía tiene un animal?

Respuesta:

Explicando por qué las interfaces no admiten atributos de instancia.

La interfaz no puede **ver** la energía, pero puede **pedirla**. La clave es que un método `default` puede llamar a otros métodos de la misma interfaz, aunque todavía no tengan código.

Cómo está escrita Mortal

```
java
```

```
public interface Mortal {
```

```
    boolean estaVivo();
```

```
    void morir();
```

```
double getEnergia(); // ← la interfaz "pide" este método
```

```
String getNombre(); // ← y este
```

```
default void verificarMuerte() {
```

```
    if (getEnergia() <= 0) {
```

```
        morir();
```

```
        System.out.println("'" + getNombre() + "' murio de inanicion");
```

```
    }
```

```
}
```

```
}
```

`verificarMuerte()` nunca toca el atributo `energia`. Llama a `getEnergia()`, que es un método que la interfaz **exige** que exista. La interfaz no sabe cómo se calcula la energía ni dónde está guardada: solo sabe que quien firme el contrato `Mortal` va a tener un método que la devuelve.

Es como un médico que no puede mirar adentro tuyo, pero te pregunta "¿cuánto pesás?" y confía en tu respuesta. No necesita tener el dato: necesita poder pedirlo.

¿Quién cumple ese contrato?

La interfaz la implementa **Animal**:

```
java
```

```
public abstract class Animal extends Entidad implements Mortal { ... }
```

Al firmar `implements Mortal`, `Animal` está obligado a tener los cuatro métodos: `estaVivo()`, `morir()`, `getEnergia()` y `getNombre()`.

- `estaVivo()` y `morir()` los escribimos en `Animal`.
- `getEnergia()` y `getNombre()` **no hubo que escribirlos**: `Animal` ya los tiene porque los hereda de `Entidad`. Java ve que existen y da el contrato por cumplido.

Qué pasa cuando se ejecuta, paso a paso

En el paso 5 del turno, el `Ecosistema` llama a `c.verificarMuerte()` para un conejo:

1. Se ejecuta el código `default` de `Mortal`.
2. Llama a `getEnergia()`. Como el objeto real es un `Conejo`, se usa el getter que el `Conejo` heredó de `Entidad`.
3. Ese getter sí puede leer el atributo `private energia`, porque es de la misma clase `Entidad`, y devuelve el valor. Por ejemplo, 0.
4. Como $0 \leq 0$, llama a `morir()`, que es el método que escribimos en `Animal`: pone `viva` en `false`.
5. Llama a `getNombre()` para armar el mensaje: `'Rex-12' murio de inanicion`.

¿Por qué no puede usar energía directamente?

Por dos motivos:

- **La interfaz no sabe que existe Entidad.** `Mortal` podría implementarla cualquier clase, así que no puede dar por hecho que tenga un atributo llamado `energia`.
- **Aunque lo supiera, el atributo es private.** Solo lo puede leer la propia clase `Entidad`. Eso es el encapsulamiento: para llegar al dato hay que pasar por el getter.

(Técnicamente, una interfaz puede tener constantes fijas, pero nunca atributos propios de cada objeto, como la energía de un conejo en particular.)